5

Instruções de controle: Parte 2

OBJETIVOS

Neste capítulo você aprenderá:

- Os princípios básicos da repetição controlada por contador.
- Como utilizar instruções de repetição for e do…while para executar instruções em um programa repetidamente.
- Como entender a seleção múltipla utilizando a instrução de seleção switch.
- Como utilizar as instruções break e continue para alterar o fluxo de controle.
- Como utilizar operadores lógicos para formar expressões condicionais complexas em instruções de controle.

Outline

5.1 Introdução

5.2 Princípios básicos de repetição controlada por contador

5.3 A instrução de repetição `for`

5.4 Exemplos com a estrutura for

5.5 A estrutura de repetição do `while`

5.6 A estrutura de seleção múltipla `switch`

5.7 Instruções `break` e `continue`

5.8 Operadores lógicos

5.9 Resumo de programação estruturada

5.1 Introdução

- **Continuação da discussão sobre programação estruturada**
  - **Introdução das estruturas de controle remanescentes em Java**
    - `for`, `do...while`, `switch`

5.2 Princípios básicos de repetição controlada por contador

- **Requer:**
  - **Variável de controle (contador de loop ou laço)**
  - **Valor inicial da variável de controle**
  - **Incremento/decremento da variável de controle a cada loop (cada iteração do laço)**
  - **Condição de continuação do laço que testa o valor final da variável de controle**

```java
// Fig. 5.1: WhileCounter.java
// Repetição controlada por contador com a instrução de repetição while.

public class WhileCounter
{
   public static void main( String args[] )
   {
      int counter = 1; // declara e inicializa a variável de controle

      while ( counter <= 10 ) // condição de continuação do laço
      {
         System.out.printf( "%d  ", counter );
         ++counter; // incrementa a variável de controle por 1
      } // fim do while

      System.out.println(); // saída de caractere de nova linha
   } // fim do main
} // fim da classe WhileCounter
```

```
1  2  3  4  5  6  7  8  9  10
```

Erro comum de programação 5.1

Uma vez que valores de ponto flutuante podem ser aproximados, controlar laços com variáveis de ponto flutuante pode resultar em valores de contador e testes de terminação imprecisos.

Dica de prevenção de erro 5.1

Controle a contagem de laços com inteiros.

Boa prática de programação 5.1

Coloque linhas em branco acima e abaixo das instruções de controle de repetição e seleção e recue os corpos da instrução para aprimorar a legibilidade.

Observação de engenharia de software 5.1

“Manter a coisa simples” permanece um bom conselho para a maior parte do código que você escreverá.

5.3 Instrução de repetição `for`

- **Especifica detalhes de repetição controlada por contador**

```java
1  // Fig. 5.2: ForCounter.java
2  // Repetição controlada por contador com a instrução de repetição for.
3
4  public class ForCounter
5  {
6     public static void main( String args[] )
7     {
8        // cabeçalho da instrução for inclui inicialização,
9        // condição de continuação do laço e incremento
10       for ( int counter = 1; counter <= 10; counter++ )
11          System.out.printf( "%d  ", counter );
12
13       System.out.println(); // imprime  caractere de nova linha
14    } // fim do main
15 } // fim da classe ForCounter
```

```
1  2  3  4  5  6  7  8  9  10
```

ForCounter.java

Linha 10
`int counter = 1;`

Linha 10
`counter <= 10;`

Linha 10
`counter++;`

Erro comum de programação 5.2

Utilizar um operador relacional incorreto ou um valor final incorreto de um contador de laço na condição de continuação do laço de uma instrução de repetição pode causar erro *off-by-one*.

Boa prática de programação 5.2

Utilizar o valor final na condição de uma instrução `while` ou `for` e utilizar o operador `<=` relacional ajuda a evitar erros *off-by-one*. Para um laço que imprime os valores de `1 a 10`, a condição de continuação do laço deve ser `counter <= 10` em vez de `counter < 10` (que causa um *erro off-by-one*) ou `counter < 11` (que é correto). Muitos programadores preferem a chamada contagem baseada em zero, em que contar 10 vezes `counter` seria inicializado como zero e o teste de continuação do laço seria `counter < 10`.

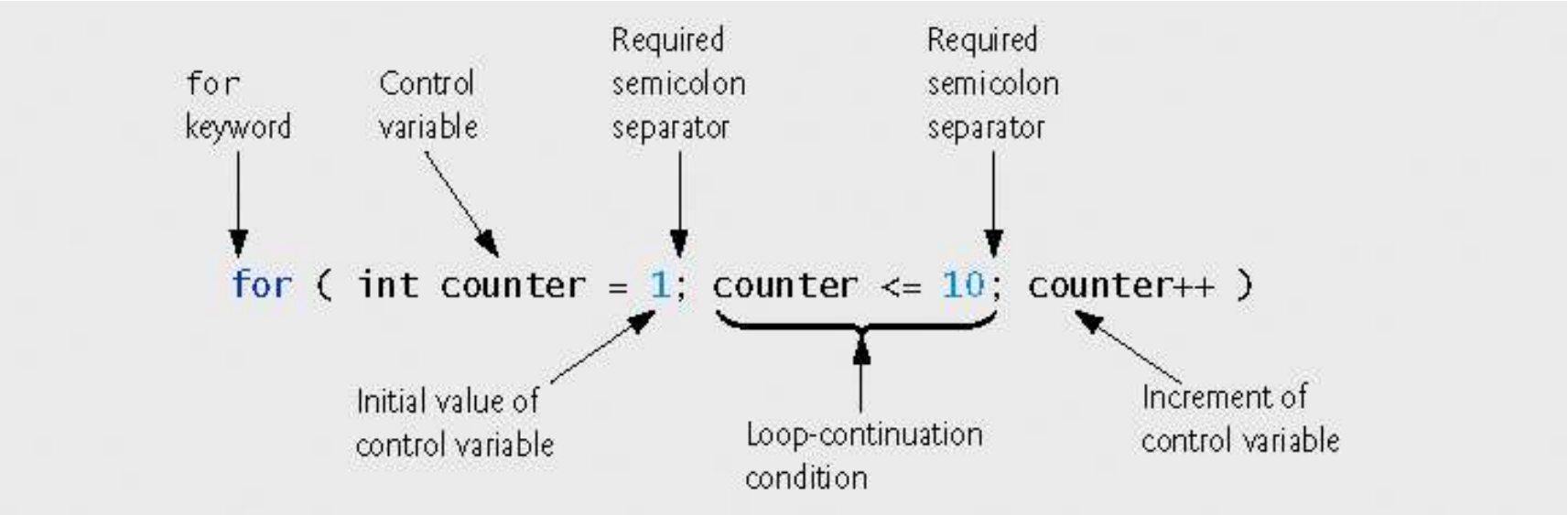

Fig. 5.3 | Componentes de cabeçalho de instrução `for`.

5.3 Instrução de repetição `for` (Cont.)

```
for ( inicialização; condiçãoDeContinuação; incremento )
    instruções;
```

Pode usualmente ser reescrita como:

```
inicialização;
  while ( condiçãoDeContinuação )
  {
      instruções;
      incremento;
  }
```

Erro comum de programação 5.3

Utilizar vírgulas em vez dos dois pontos-e-vírgulas obrigatórios em um cabeçalho `for` é um erro de sintaxe.

Erro comum de programação 5.4

Quando a variável de controle de uma instrução `for` for declarada na seção de inicialização do cabeçalho do `for`, utilizar a variável de controle depois do corpo do `for` é um erro de compilação.

Dica de desempenho 5.1

Há uma ligeira vantagem de desempenho em pré-incrementar, mas se você escolher pós-incrementar porque parece mais natural (como em um cabeçalho `for`), otimizar os compiladores vai gerar o bytecode Java que, de todo modo, utiliza a forma mais eficiente.

Boa prática de programação 5.3

Na maioria dos casos, tanto a pré-incrementação quanto a pós-incrementação são utilizadas para adicionar 1 a uma variável em uma instrução por si mesma. Nesses casos, o efeito é exatamente o mesmo, exceto pelo fato de que pré-incrementar tem uma ligeira vantagem de desempenho. Dado que o compilador em geral otimiza seu código para ajudá-lo a obter o melhor desempenho, utilize o idioma com o qual você se sente mais à vontade nessas situações.

Erro comum de programação 5.5

Colocar um ponto-e-vírgula imediatamente à direita do parêntese direito de um cabeçalho `for` torna o corpo desse `for` uma instrução vazia. Normalmente esse é um erro de lógica.

Dica de prevenção de erro 5.2

Os laços infinitos ocorrem quando a condição de continuação do laço em uma instrução de repetição nunca se torne `false`. Para evitar essa situação em um laço controlado por contador, assegure que a variável de controle é incrementada (ou decrementada) durante cada iteração do laço. Em um laço controlado por sentinela, certifique-se de que o valor da sentinela seja por fim inserido.

Dica de prevenção de erro 5.3

Embora o valor da variável de controle possa ser alterado no corpo de um laço `for`, evite fazê-lo assim porque essa prática pode levar a erros sutis.

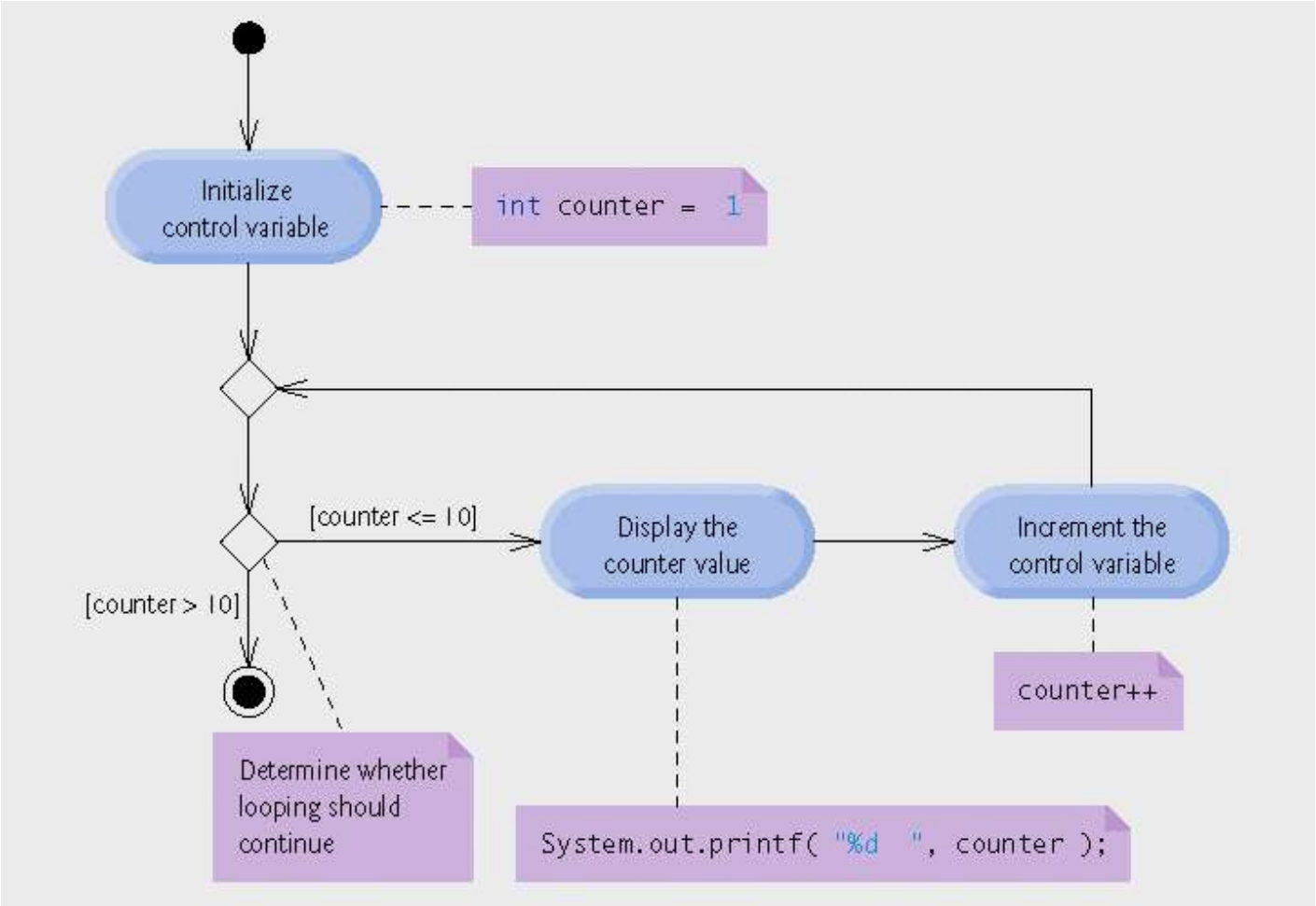

Fig. 5.4 | Diagrama de atividade UML para a instrução `for` na Fig. 5.2.

5.4 Exemplos com o `for`

- **Variando a variável de controle na instrução `for`**
  - **Varia a variável de controle de 1 a 100 em incrementos de 1**
    - `for ( int i = 1; i <= 100; i++ )`
  - **Varia a variável de controle de 100 a 1 em incrementos de –1**
    - `for ( int i = 100; i >= 1; i-- )`
  - **Varia a variável de controle de 7 a 77 em incrementos de 7**
    - `for ( int i = 7; i <= 77; i += 7 )`
  - **Varia a variável de controle de 20 a 2 em decrementos de 2**
    - `for ( int i = 20; i >= 2; i -= 2 )`
  - **Varia a variável de controle sobre a seqüência: 2, 5, 8, 11, 14, 17, 20**
    - `for ( int i = 2; i <= 20; i += 3 )`
  - **Varia a variável de controle sobre a seqüência: 99, 88, 77, 66, 55, 44, 33, 22, 11, 0**
    - `for ( int i = 99; i >= 0; i -= 11 )`

Erro comum de programação 5.6

Não utilizar o operador relacional adequado na condição de continuação de um laço que conta para baixo (por exemplo, utilizar `i <= 1` em vez de `i >= 1` em uma contagem de laço para baixo até 1) normalmente é um erro de lógica.

```java
// Fig. 5.5: Sum.java
// Somando inteiros com a instrução for.

public class Sum
{
   public static void main( String args[] )
   {
      int total = 0; // inicialize total

      // total de inteiros pares de 2 a 20
      for ( int number = 2; number <= 20; number += 2 )
         total += number;

      System.out.printf( "A soma é %d\n", total ); // exibe resultados
   } // fim do main
} // fim da classe Sum
```

```
A soma é 110
```

5.4 Exemplos com o `for` (Cont.)

- **Expressões de inicialização e incremento podem ser listas separadas por vírgulas**
  - **Por exemplo, linhas 11-12 da Fig. 5.5 pode ser reescrita como**

```
for ( int number = 2; number <= 20; total += number, number += 2 )
   ;  // instrução vazia
```

Boa prática de programação 5.4

Limite o tamanho de cabeçalhos de instrução de controle a uma única linha, se possível.

Boa prática de programação 5.5

Coloque apenas expressões que envolvem as variáveis de controle nas seções de inicialização e incremento de uma instrução `for`. As manipulações de outras variáveis devem aparecer antes do laço (se executarem apenas uma vez, como as instruções de inicialização) ou no corpo do laço (se executarem uma vez por iteração do laço, como as instruções de incremento ou decremento).

```java
// Fig. 5.6: Interest.java
// Cálculos de juros compostos com for.

public class Interest
{
   public static void main( String args[] )
   {
      double amount; // quantia em depósito ao fim de cada ano
      double principal = 1000.0; // quantia inicial antes dos juros
      double rate = 0.05; // taxa de juros

      // exibe cabeçalhos
      System.out.printf( "%s%20s\n", "Ano", "Quantia em depósito" );

```

```
15        // calcula quantia de depósito para os dez anos
16        for ( int year = 1; year <= 10; year++ )
17        {
18           // calcula nova quantia durante ano especificado
19           amount = principal * Math.pow( 1.0 + rate, year );
20
21           // exibe o ano e a quantia
22           System.out.printf( "%4d%,20.2f\n", year, amount );
23        } // fim do for
24     } // fim do main
25  } // fim da classe Interest
```

```
Ano   Quantia em depósito
   1            1,050.00
   2            1,102.50
   3            1,157.63
   4            1,215.51
   5            1,276.28
   6            1,340.10
   7            1,407.10
   8            1,477.46
   9            1,551.33
  10            1,628.89
```

5.4 Exemplos com o `for` (Cont.)

- **Saída formatada**
  - **Largura de campo**
  - **Flag de formatação do sinal de subtração (`-`)**
  - **Vírgula (`,`) flag de formatação**
- **Método `static`**
  - ***NomeDaClasse.nomeDoMetodo*(*argumentos*)**

Boa prática de programação 5.6

Nao utilizar variáveis de tipo `double` (ou `float`) para realizar cálculos monetários precisos. A imprecisão de números de ponto flutuante pode causar erros que resultarão em valores monetários incorretos. Nos exercícios, exploramos o uso de inteiros para realizar cálculos monetários. [Nota: Alguns fornecedores independentes vendem bibliotecas de classe que realizam cálculos monetários precisos. Além disso, a API do Java fornece a classe `java.math.BigDecimal` para realizar cálculos com valores de ponto flutuante de precisão arbitrária.]

Dica de desempenho 5.2

Em laços, evite cálculos para os quais o resultado nunca muda — esses cálculos em geral devem ser colocados antes do laço. [Nota: Muitos compiladores de otimização sofisticados atuais colocarão esses cálculos for a de laços no código compilado.]

5.5 Instrução de repetição `do...while`

- **Instrução `do...while`**
  - **Semelhante à instrução `while`**
  - **Testa a condição de continuação do laço depois de executar o corpo do laço**
    - **i.e., o corpo do laço executa pelo menos uma vez**

```java
// Fig. 5.7: DoWhileTest.java
// Instrução de repetição do...while.

public class DoWhileTest
{
   public static void main( String args[] )
   {
      int counter = 1; // inicialize counter

      do
      {
         System.out.printf( "%d  ", counter );
         ++counter;
      } while ( counter <= 10 ); // fim do do...while

      System.out.println(); // pula uma linha
   } // fim do main
} // fim da classe DoWhileTest
```

```
1  2  3  4  5  6  7  8  9  10
```

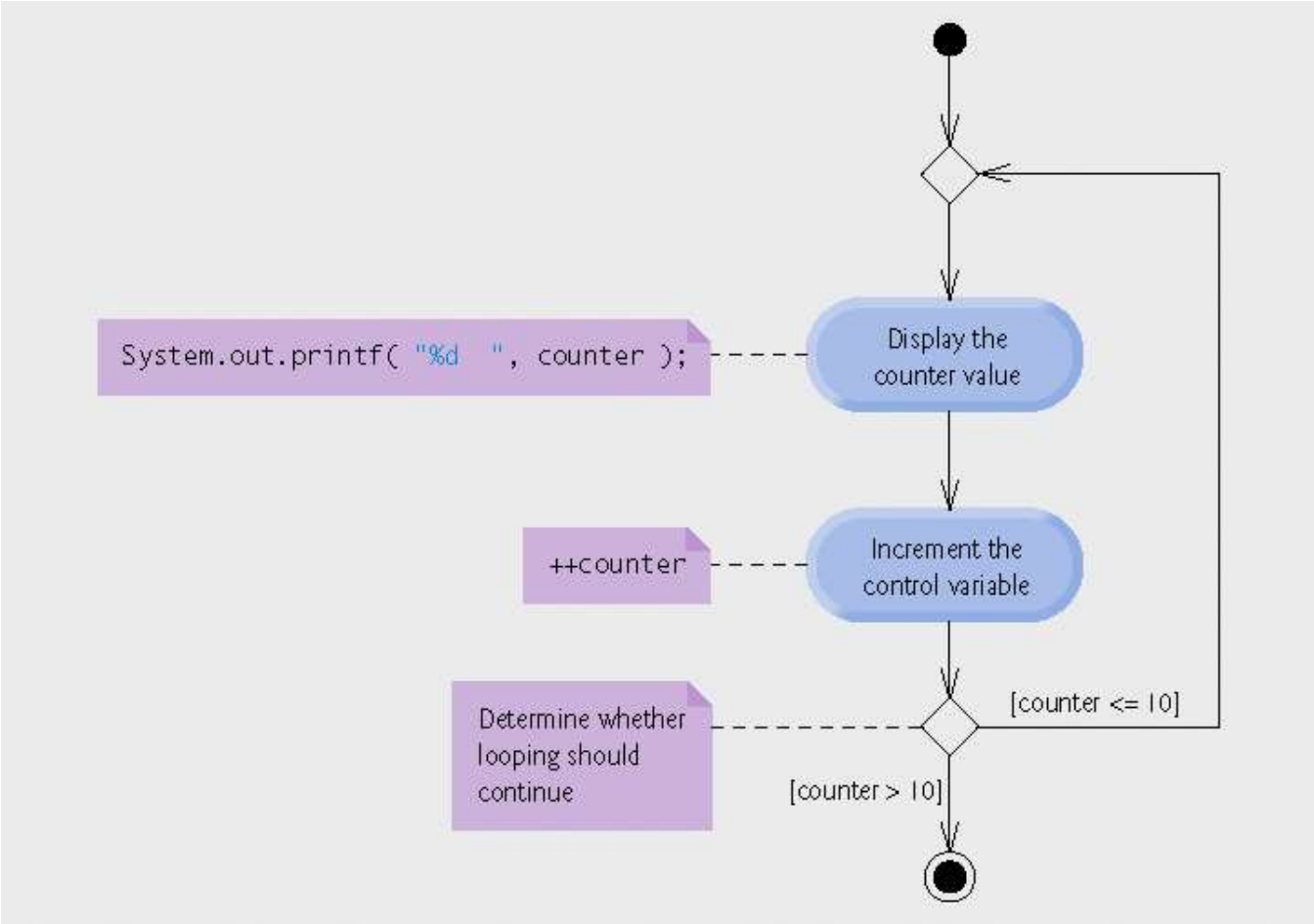

Fig. 5.8 | Diagrama de atividades UML da instrução de repetição do...while.

Boa prática de programação 5.7

Sempre inclua chaves em uma instrução `do...while`, mesmo se não forem necessárias. Isso ajuda a eliminar ambigüidade entre a instrução `while` e uma instrução `do...while` que contém apenas uma instrução.

5.6 Estrutura de seleção múltipla `switch`

- **Estrutura `switch`**
  - **Utilizada para seleções múltiplas**

```java
// Fig. 5.9: GradeBook.java
// Classe que utiliza switch para contar as notas A, B, C, D and F.
import java.util.Scanner; // programa usa classe Scanner

public class GradeBook
{
   private String nomeDoCurso; // nome do curso
   private int total; // soma de notas
   private int contNotas; // número de notas inseridas
   private int aCount; // contagem de notas A
   private int bCount; // contagem de notas B
   private int cCount; // contagem de notas C
   private int dCount; // contagem de notas D
   private int fCount; // contagem de notas F

   // construtor inicializa nomeDoCurso;
   // variáveis de instância int são inicializadas como 0 por padrão
   public GradeBook( String name )
   {
      nomeDocurso = name; // inicializa nomeDoCurso
   } // fim do construtor

   // método para configurar o nome do curso
   public void setNomeDoCurso ( String name )
   {
      nomeDoCurso = name; // armazena o nome do curso
   } // fim do método setNomeDoCurso

```

```
29    // método para recuperar o nome do curso
30    public String getNomeDoCurso()
31    {
32       return nomeDoCurso;
33    } // fim do método getNomeDoCurso
34
35    // exibe uma mensagem
36    public void displayMessage()
37    {
38       // getNomeDoCurso obtém o nome do curso
39       System.out.printf( "Bem-vindo à disciplina\n%s!\n\n",
40          getNomeDoCurso() );
41    } // fim do método displayMessage
42
43    // insere número arbitrário de notas do usuário
44    public void inputGrades()
45    {
46       Scanner input = new Scanner( System.in );
47
48       int grade; // nota inserida pelo usuário
49
50       System.out.printf( "%s\n%s\n   %s\n   %s\n",
51          "Insira notas inteiras no intervalo de 0-100.",
52          "Digite o indicador de fim de arquivo para finalizar a
entrada:",
53          "Em UNIX/Linux/Mac OS X digite <ctrl> d então pressione Enter",
54          "Em Windows digite <ctrl> z então pressione Enter" );
55
```

```
56        // faz o laço até a variável de sentinela
57        while ( input.hasNext() )
58        {
59           grade = input.nextInt(); // lê a nota
60           total += grade; // adiciona grade a total
61           ++contNotas; // incrementa o número de notas
62
63           // chama o método para incrementar o contador adequado
64           incrementaConceitos( grade );
65        } // fim do while
66     } // fim do método inputGrades
67
68     // adiciona 1 ao contador adequado da nota especificada
69     public void incrementaConceitos( int valor )
70     {
71        // determina a nota que foi inserida
72        switch ( valor / 10 )
73        {
74           case 9:  // nota estava entre 90
75           case 10: // e 100
76              ++aCount; // incrementa aCount
77              break; // necessário para sair do switch
78
79           case 8: // nota estava entre 80 e 89
80              ++bCount; // incrementa bCount
81              break; // sai do switch
82
```

```
83          case 7: // nota estava entre 70 e 79
84             ++cCount; // incrementa cCount
85             break; // sai do switch
86
87          case 6: // nota estava entre 60 e 69
88             ++dCount; // incrementa dCount
89             break; // sai do switch
90
91          default: // nota menor que 60
92             ++fCount; // incrementa fCount
93             break; // opcional; sai do switch de qualquer maneira
94       } // fim do switch
95    } // fim do método incrementaConceitos
96
97    // exibe relatório baseado nas notas inseridas
98    public void displayGradeReport()
99    {
100        System.out.println( "\nRelatório de notas:" );
101
102        // se usuário inseriu pelo menos uma nota...
103        if ( contNotas != 0 )
104        {
105           // calcula a média de todas as notas inseridas
106           double media = (double) total / contNotas;
107
```

```
108         // gera a saída
109         System.out.printf( "Total de %d notas inseridas é %d\n",
110            contNotas, total );
111         System.out.printf( "Média da turma é %.2f\n", media );
112         System.out.printf( "%s\n%s%d\n%s%d\n%s%d\n%s%d\n%s%d\n",
113            "Número de alunos por conceito:",
114            "A: ", aCount,   // exibe número de notas A
115            "B: ", bCount,   // exibe número de notas B
116            "C: ", cCount,   // exibe número de notas C
117            "D: ", dCount,   // exibe número de notas D
118            "F: ", fCount ); // exibe número de notas F
119      } // fim do if
120      else // nehuma nota foi inserida, outra mensagem
121         System.out.println( "Sem notas inseridas!" );
122   } // fim do método displayGradeReport
123 } // fim da classe GradeBook
```

Dica de portabilidade 5.1

As combinações de pressionamento de teclas para inserir o fim de arquivo são dependentes de sistema.

Erro comum de programação 5.7

Esquecer uma instrução *break* quando esta for necessária em um *switch* é um erro de lógica.

```java
// Fig. 5.10: GradeBookTest.java
// Cria objeto GradeBook, insere notas e exibe o relatório de notas.

public class GradeBookTest
{
   public static void main( String args[] )
   {
      // cria objeto g1 da classe GradeBook e
      // passa o nome do curso ao construtor
      GradeBook g1 = new GradeBook(
         "Algoritmos e Linguagem de Programação II" );

      g1.displayMessage(); // exibe mensagem de boas-vindas
      g1.inputGrades(); // lê notas digitadas pelo usuário
      g1.displayGradeReport(); // exibe relatório baseado nas notas
   } // fim do main
} // fim da classe GradeBookTest
```

```
Bem-vindo à disciplina
Algoritmos e Linguagem de Programação II!

Insira notas inteiras no intervalo 0-100.
Digite o indicador de fim de arquivo para finalizar a entrada:
   Em UNIX/Linux/Mac OS X digite <ctrl> d então pressione Enter
   Em Windows digite <ctrl> z então pressione Enter
99
92
45
57
63
71
76
85
90
100
^Z

Relatório de notas:
Total de 10 notas inseridas é 778
Média da classe é 77.80
Número de alunos por conceito:
A: 4
B: 1
C: 2
D: 1
F: 2
```

Observação de engenharia de software 5.2

Forneça uma instrução *default* em um *switch*. Incluir uma instrução *default* faz com que você se concentre na necessidade de processar condições excepcionais.

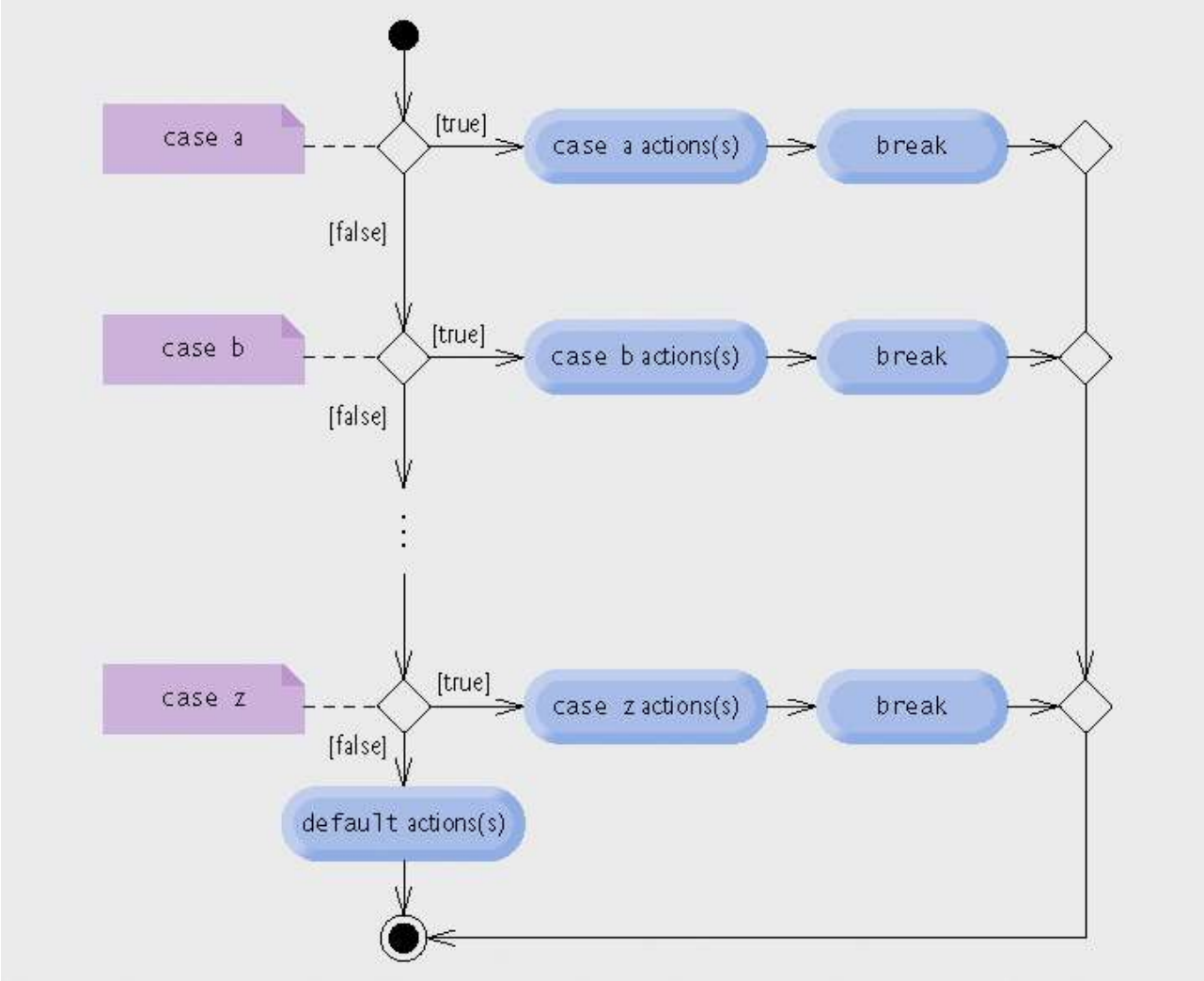

Fig. 5.11 | Diagrama de atividade UML de instrução de seleção múltipla *switch* com instruções *break*.

Boa prática de programação 5.8

Embora cada *case* e a instrução *default* em um *switch* possam ocorrer em qualquer ordem, coloque a instrução *default* por último. Quando a instrução *default* é listada por último, o *break* para essa instrução não é necessário. Alguns programadores incluem este *break* para clareza e simetria com outros *cases*.

5.6 Estrutura de seleção múltipla `switch` (Cont.)

- **Expressão em cada `case`**
  - **Expressão integral constante**
    - **Combinação de constantes inteiras avaliam um valor inteiro constante**
  - **Constante de caractere**
    - **P.ex., ‘A’, ‘7’ ou ‘$’**
  - **Variável constante**
    - **Declarada com a palavra-chave `final`**

5.7 Instruções `break` e `continue`

- `break/continue`
  - Alteram o fluxo de controle do programa
- Instrução `break`
  - Causa a saída imediata da estrutura de controle
    - Utilizado em instruções `while`, `for`, `do…while` ou `switch`
- Instrução `continue`
  - Pula as instruções restantes no corpo do laço
  - Prossegue com a próxima iteração
    - Utilizado em instruções `while`, `for` ou `do…while`

```java
// Fig. 5.12: BreakTest.java
// Instrução break sai de uma instrução for.
public class BreakTest
{
   public static void main( String args[] )
   {
      int count; // variável de controle também usada depois do laço

      for ( count = 1; count <= 10; count++ ) // o laço 10 vezes
      {
         if ( count == 5 ) // se count for 5,
            break;         // finaliza o laço

         System.out.printf( "%d ", count );
      } // fim do for

      System.out.printf( "\nSai do laço quando count = %d\n", count );
   } // fim do main
} // fim da classe BreakTest
```

```
1 2 3 4
Sai do laço quando count = 5
```

```
1  // Fig. 5.13: ContinueTest.java
2  // instrução continue termina uma iteração de uma instrução for.
3  public class ContinueTest
4  {
5     public static void main( String args[] )
6     {
7        for ( int count = 1; count <= 10; count++ ) // laço 10 vezes
8        {
9           if ( count == 5 ) // se count for 5,
10             continue;      // pula o código restante no laço
11
12          System.out.printf( "%d ", count );
13       } // fim do for
14
15       System.out.println( "\n Instrução continue usada para pular
impressão do 5" );
16    } // fim do main
17 } // fim da classe ContinueTest
```

Skip lin
line 7 w

```
1 2 3 4 6 7 8 9 10
Instrução continue usada para pular impressão do 5
```

Observação de engenharia de software 5.3

Alguns programadores consideram que <u>break</u> e <u>continue</u> violam a programação estruturada. Visto que os mesmos efeitos são alcançáveis com as técnicas de programação estruturada, esses programadores não utilizam <u>break</u> e <u>continue</u>.

Observação de engenharia de software 5.4

Há uma tensão entre alcançar engenharia de software de qualidade e alcançar o software de melhor desempenho. Freqüentemente, um desses objetivos é alcançado à custa do outro. Para todas as situações exceto as de desempenho muito alto, aplique a seguinte regra geral: Primeiro, faça seu código simples e correto; então, torne-o rápido e pequeno, mas apenas se necessário.

5.8 Operadores lógicos

- **Operadores lógicos**
  - **Permitem formar condições mais complexas**
  - **Combinação de condições simples**
- **Operadores lógicos em Java**
  - **&&** **(E condicional)**
  - **||** **(OU condicional)**
  - **&** **(E lógico booleano)**
  - **|** **(OU inclusivo lógico booleano)**
  - **^** **(OU exclusivo lógico booleano)**
  - **!** **(NÃO lógico)**

5.8 Operadores lógicos (Cont.)

- **Operador E condicional (&&)**
  - **Considere a seguinte instrução `if`**

```
if ( sexo == FEM && idade >= 65 )
   ++seniorFem;
```

  - **Condição combinada é `true`**
    - **Se e somente se as condições simples forem `true`**
  - **Condição combinada é `false`**
    - **Se qualquer uma ou ambas as condições simples forem `false`**

expressão1	expressão2	expressão1 && expressão2
false	false	false
false	true	false
true	false	false
true	true	true

Fig. 5.14 | Tabela-verdade do operador && (E condicional) .

5.8 Operadores lógicos (Cont.)

- **Operador OU condicional (`||`)**
  - **Considere a seguinte instrução `if`**

```
if ( ( mediaSemestre >= 90 ) || ( ultimaProva >= 90 )
   System.out.println( “Conceito do aluno é A” );
```

  - **Condição combinada é `true`**
    - **Se qualquer uma ou ambas as condições simples forem `true`**
  - **Condição combinada é `false`**
    - **Se ambas as condições simples forem `false`**

expressão1	expressão2	expressão1 | | expressão2
false	false	false
false	true	true
true	false	true
true	true	true

Fig. 5.15 | Tabela-verdade do operador | | (OU condicional).

5.8 Operadores lógicos (Cont.)

- **Avaliação de curto-circuito de condições complexas**
  - **As partes de uma expressão contendo operadores && ou || são avaliadas somente até que se saiba se a condição é verdadeira ou falsa**
  - **P.ex.,** `(sexo == FEM ) && ( idade >= 65 )`
    - **Para imediatamente se a variável sexo não é igual a FEM**

Erro comum de programação 5.8

Em expressões que utilizam o operador &&, uma condição – que chamaremos de condição <u>dependente</u> – pode exigir que outra condição seja verdadeira para que a avaliação da condição dependente tenha significado. Neste caso, a condição dependente deve ser colocada depois da outra condição, ou um erro pode ocorrer. Por exemplo, na expressão `(i != 0) && (10 / i == 2)`, a segunda condição deve aparecer depois que a primeira condição ou pode ocorrer um erro de divisão por zero.

5.8 Operadores lógicos (Cont.)

- **Operador E lógico booleano (&)**
  - **Funciona de modo idêntico ao &&**
  - **Exceto que & sempre avalia ambos os operandos**
- **Operador OU lógico booleano (|)**
  - **Funciona de modo idêntico ao ||**
  - **Exceto que | sempre avalia ambos os operandos**

Dica de prevenção de erro 5.4

Por questão de clareza, evite expressões com efeitos colaterais nas condições. Os efeitos colaterais talvez pareçam inteligentes, mas podem dificultar o entendimento do código e podem levar a erros de lógica sutis.

5.8 Operadores lógicos (Cont.)

- **OU exclusivo lógico booleano (^)**
  - **Um de seus operandos é `true` e o outro é `false`**
    - **Avalia como `true`**
  - **Ambos os operandos são `true` ou ambos são `false`**
    - **Avalia como `false`**
- **Operador de negação lógica (`!`)**
  - **Operador unário**

expressão1	expressão2	expressão1 ^ expressão2
false	false	false
false	true	true
true	false	true
true	true	false

Fig. 5.16 | Tabela-verdade do operador ^ (OU exclusivo lógico booleano).

expressão	!expressão
`false`	`true`
`true`	`false`

Fig. 5.17 |Tabela-verdade do operador !(negação lógica ou NÃO lógico).

```java
// Fig. 5.18: LogicalOperators.java
// Operadores Lógicos.

public class LogicalOperators
{
   public static void main( String args[] )
   {
      // cria tabela-verdade para o operador && (E condicional)
      System.out.printf( "%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
         "E Condicional (&&)", "false && false", ( false && false ),
         "false && true", ( false && true ),
         "true && false", ( true && false ),
         "true && true", ( true && true ) );

      // cria tabela-verdade para o operador || (OU condicional)
      System.out.printf( "%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
         "OU Condicional (||)", "false || false", ( false || false ),
         "false || true", ( false || true ),
         "true || false", ( true || false ),
         "true || true", ( true || true ) );

      // cria tabela-verdade para o operador & (E lógico booleano)
      System.out.printf( "%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
         "E lógico booleano (&)", "false & false", ( false & false ),
         "false & true", ( false & true ),
         "true & false", ( true & false ),
         "true & true", ( true & true ) );

```

```
29        // cria tabela para o operador | (OU inclusivo lógico booleano)
30        System.out.printf( "%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
31           "OU inclusivo lógico booleano (|)",
32           "false | false", ( false | false ),
33           "false | true", ( false | true ),
34           "true | false", ( true | false ),
35           "true | true", ( true | true ) );
36
37        // cria tabela para o operador ^ (OU exclusivo lógico booleano)
38        System.out.printf( "%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
39           "OU exclusivo lógico booleano (^)",
40           "false ^ false", ( false ^ false ),
41           "false ^ true", ( false ^ true ),
42           "true ^ false", ( true ^ false ),
43           "true ^ true", ( true ^ true ) );
44
45        // cria tabela para o operador ! (negação lógica)
46        System.out.printf( "%s\n%s: %b\n%s: %b\n", "NÃO lógico (!)",
47           "!false", ( !false ), "!true", ( !true ) );
48     } // fim do main
49  } // fim da classe LogicalOperators
```

```
E Condicional (&&)
false && false: false
false && true: false
true && false: false
true && true: true

OU Condicional (||)
false || false: false
false || true: true
true || false: true
true || true: true

E lógico booleano (&)
false & false: false
false & true: false
true & false: false
true & true: true

OU inclusivo lógico booleano (|)
false | false: false
false | true: true
true | false: true
true | true: true

OU exclusivo lógico booleano (^)
false ^ false: false
false ^ true: true
true ^ false: true
true ^ true: false

NÃO lógico (!)
!false: true
!true: false
```

Operadores	Associatividade	Tipo
++ --	da direita para a esquerda	unário pós-fixo
++ - + - ! (*tipo*)	da direita para a esquerda	unário pré-fixo
* / %	da esquerda para a direita	multiplicativo
+ -	da esquerda para a direita	aditivo
< <= > >=	da esquerda para a direita	relacional
== !=	da esquerda para a direita	igualdade
&	da esquerda para a direita	E lógico booleano
^	da esquerda para a direita	OU exclusivo lógico booleano
|	da esquerda para a direita	OU inclusivo lógico booleano
&&	da esquerda para a direita	E condicional
||	da esquerda para a direita	OU condicional
?:	da direita para a esquerda	condicional
= += -= *= /= %=	da direita para a esquerda	atribuição

Fig. 5.19 | Precedência/associatividade dos operadores discutidos até agora.

5.9 Resumo de programação estruturada

- **Estrutura de seqüência**
  - **“Embutida” no Java**
- **Estrutura de seleção**
  - `if`, `if…else` e `switch`
- **Estrutura de repetição**
  - `while`, `do…while` e `for`

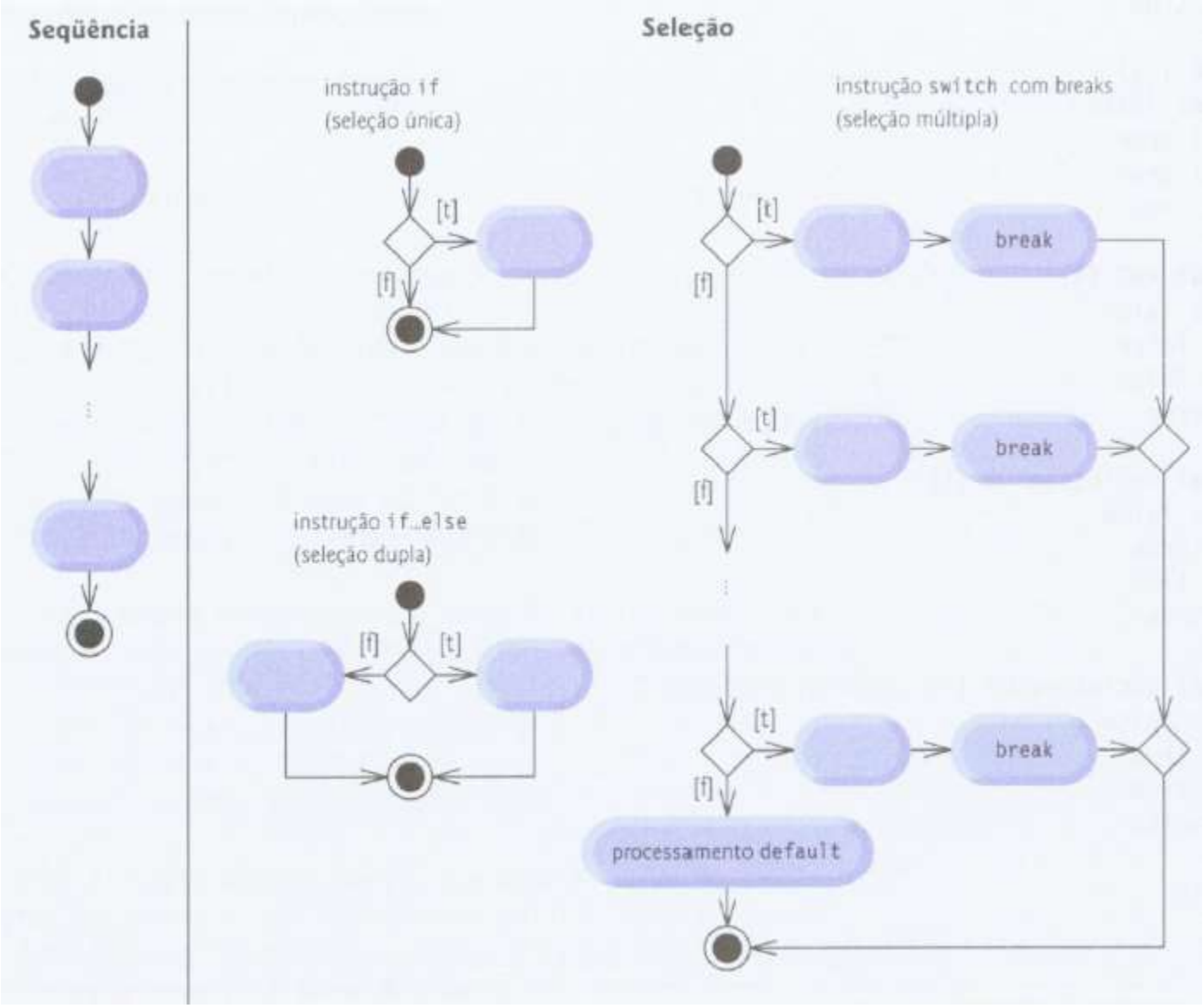

Fig. 5.20 | Instruções de seqüência de entrada única/saída única, seleção e repetição do Java (1 de 2).

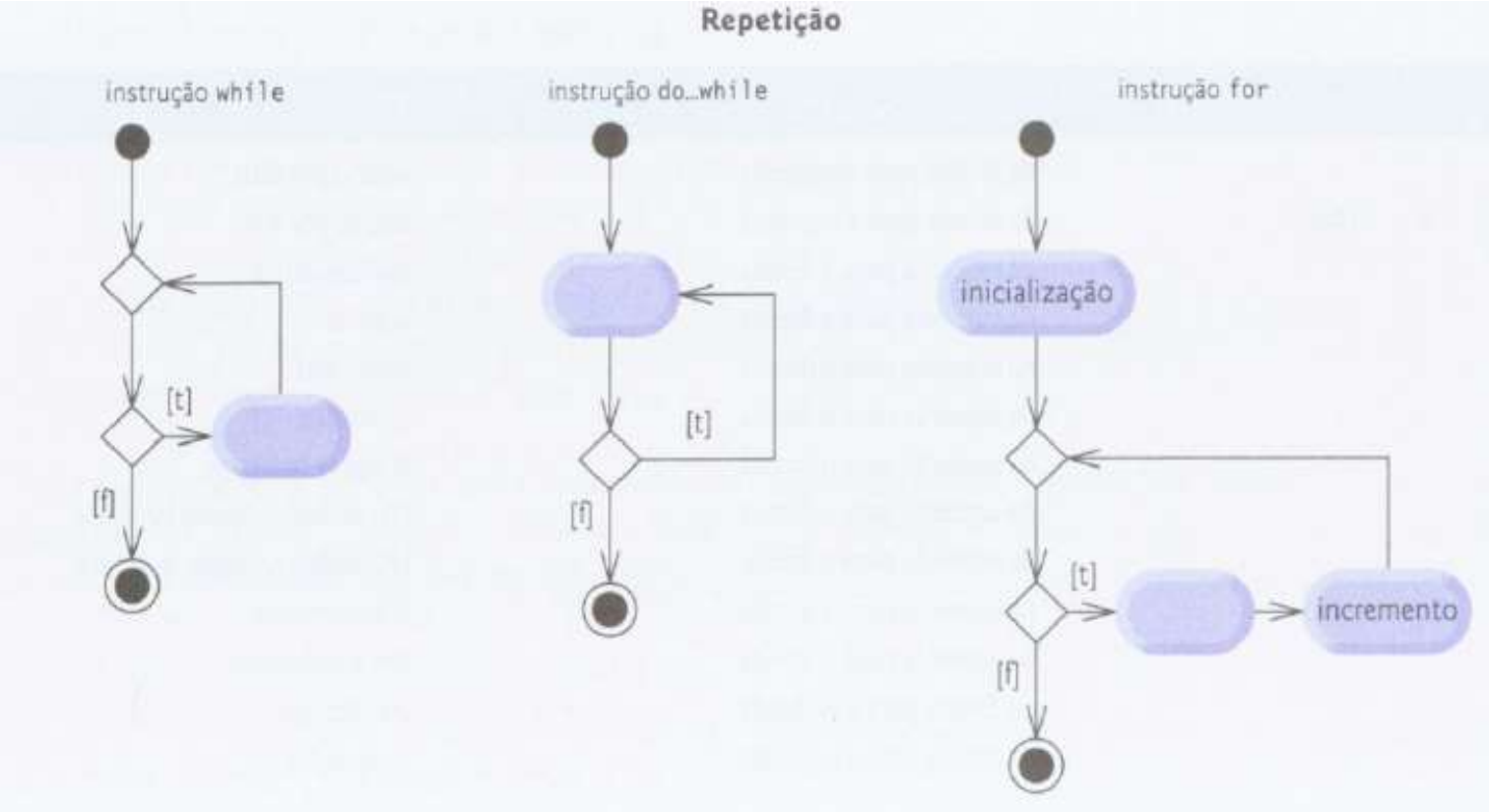

Fig. 5.20 | Instruções de seqüência de entrada única/saída única, seleção e repetição do Java (2 de 2).

Regras para formar programas estruturados	
1	Comece com o diagrama de atividade mais simples (Fig. 5.22).
2	Qualquer estado de ação pode ser substituído por dois estados de ação em seqüência.
3	Qualquer estado de ação pode ser substituído por qualquer instrução de controle (seqüência de estados de ação, `if`, `if else`, `switch`, `while`, `do while` ou `for`).
4	Regas 2 e 3 podem ser aplicadas com a freqüência que você quiser e em qualquer ordem.

Fig. 5.21 | Regras para formar programas estruturados.

Fig. 5.22 | Diagrama de atividade mais simples.

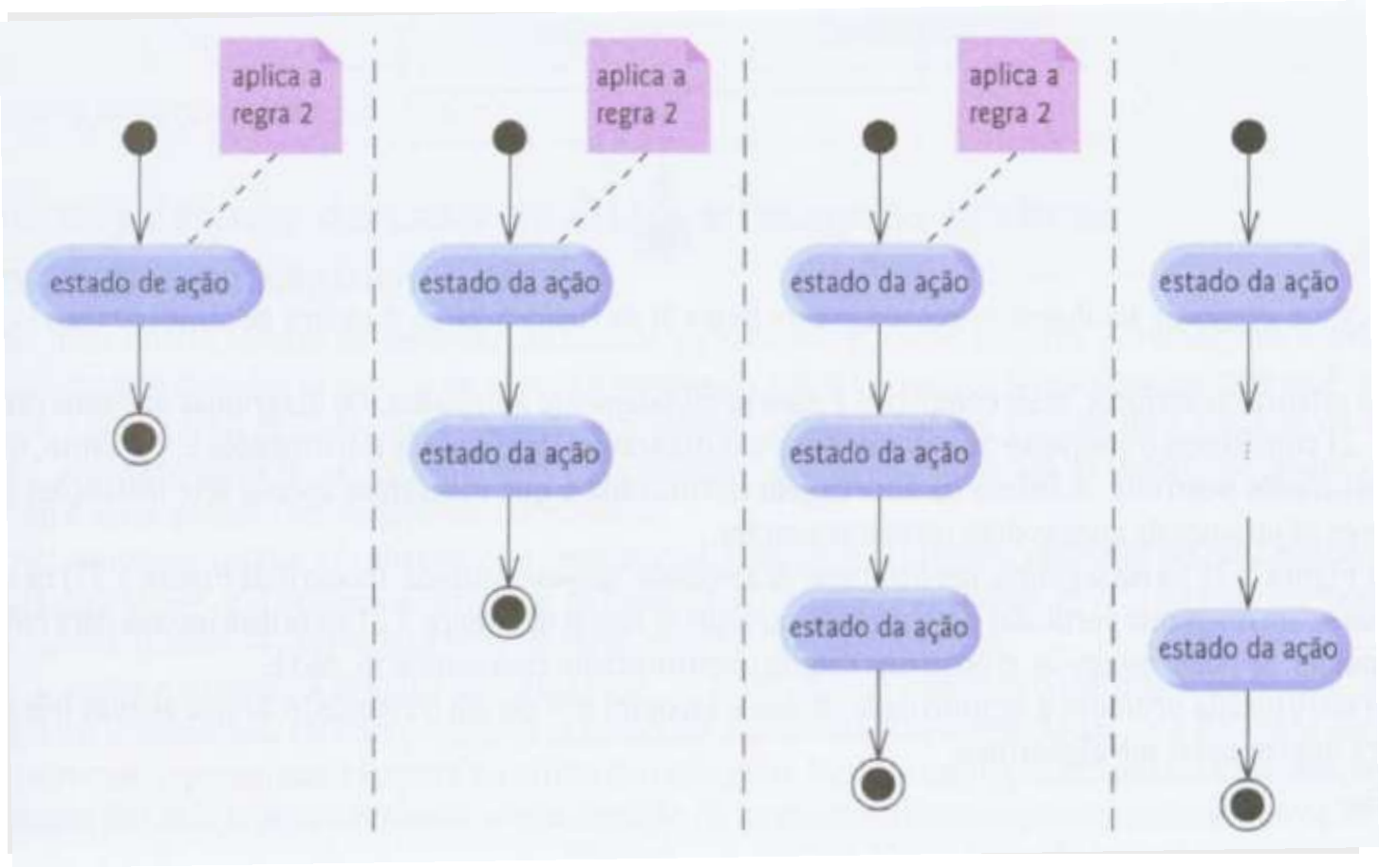

Fig. 5.23 | Aplicando a regra de empilhamento repetidamente (regra 2) da Fig. 5.21 ao diagrama de atividade mais simples.

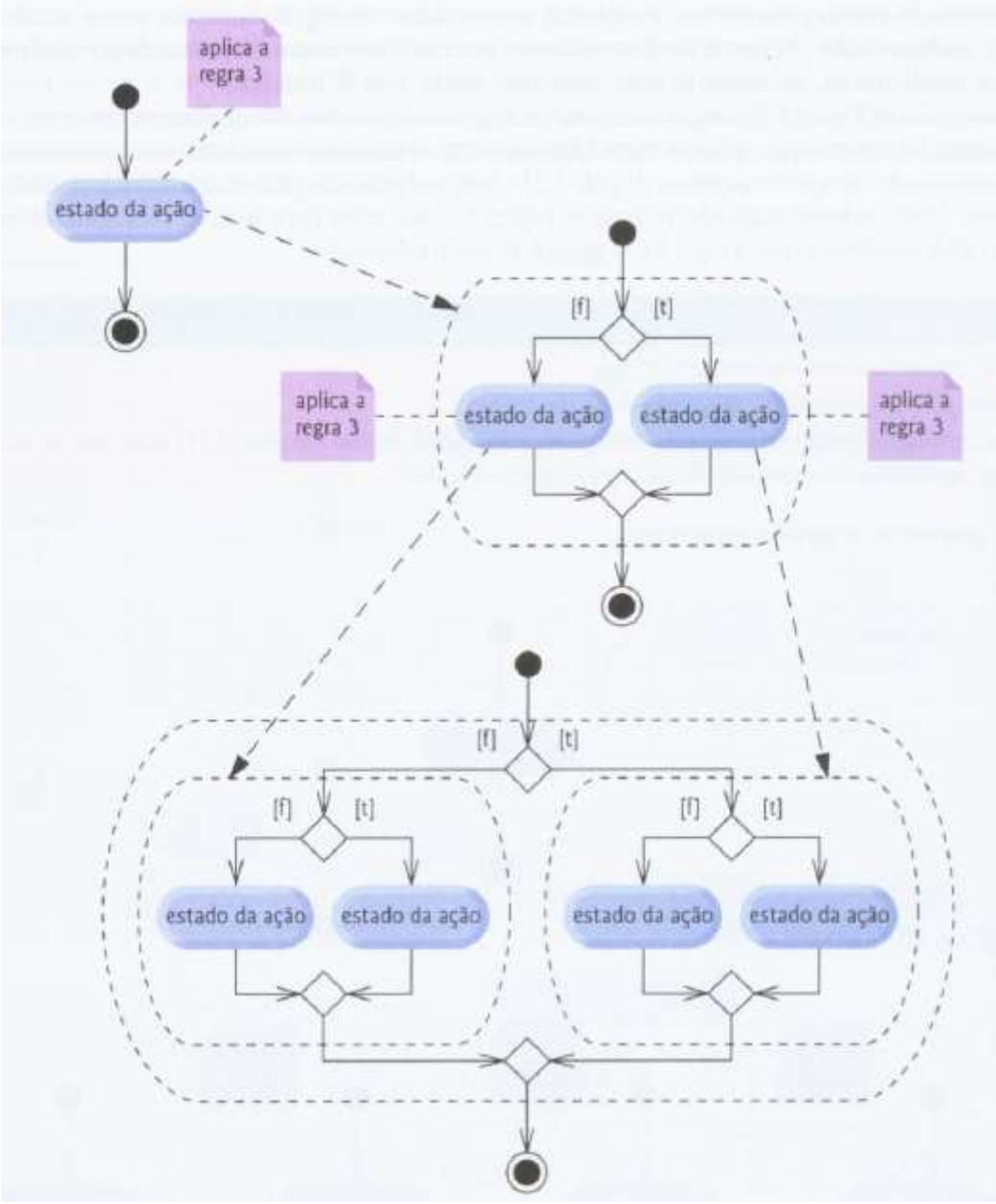

Fig. 5.24 | Aplicando a regra de aninhamento repetidamente (regra 3) da Fig. 5.21 ao diagrama de atividade mais simples.

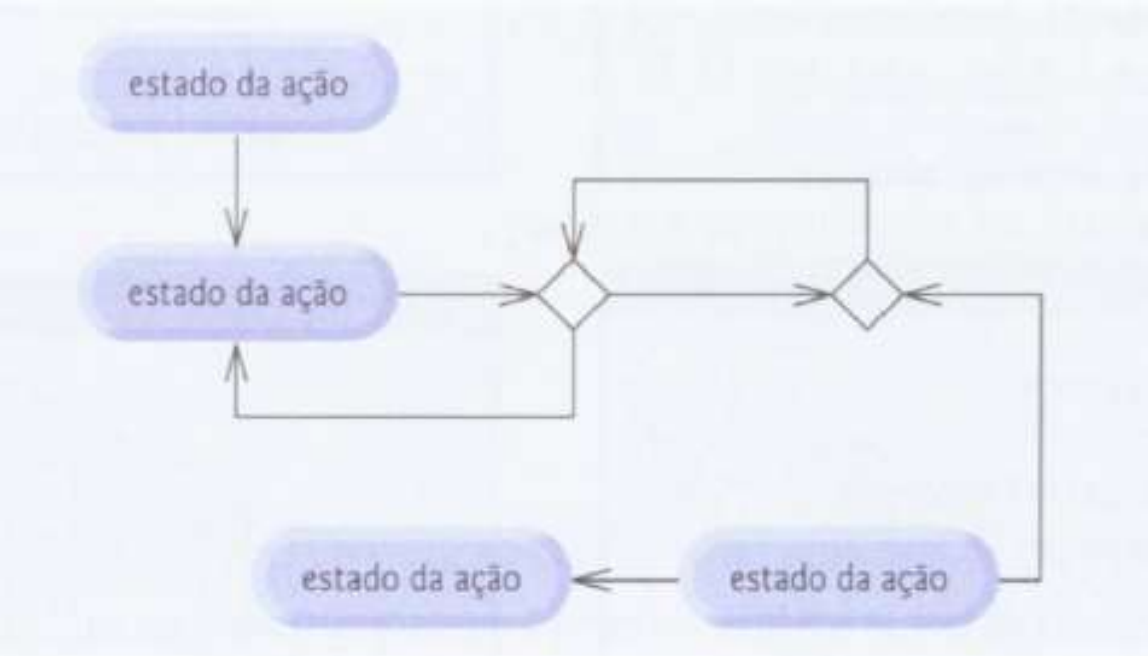

Fig. 5.25 | **Diagrama de atividade 'não estruturado'.**

Referências

Adaptado de: “Teaching Resources for Java How to Program, 6/e”.

Deitel & Deitel. Java como programar, 6 ed.

Kathy Sierra e Bert Bates. Head First Java!.

Notas de aula da disciplina Projeto com Orientação a Objetos.